

SOCRATES для ИИ

через BOIS: SIMA & BORIS

Как программировать ИИ-продукты физиологией, а не только промптом



Обновленная учебная статья: теперь с SIMA&BORIS, практикой Сократ-программирования и примерами для ИИ-продуктов.

1. Зачем нужна эта статья

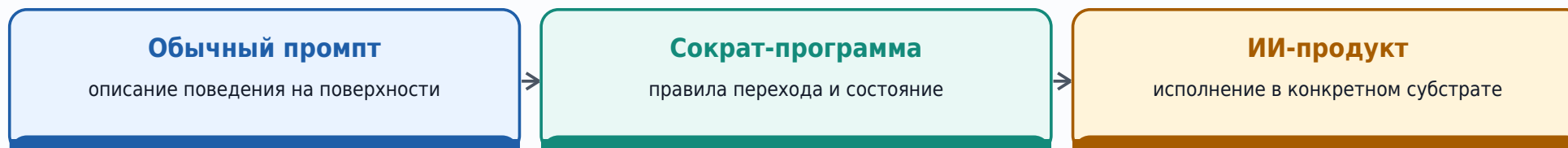
Первая версия объясняла SOCRATES/COKPAT как подход к организации расчета, но была слишком общей. В ней не хватало главного практического слоя: как BOIS используется через SIMA&BORIS и как из этого получается программирование ИИ-продуктов.

! Главная мысль

SOCRATES/COKPAT - это не "хороший промпт". Это способ описать физиологию расчета: область, ценности, субстрат, правила, состояние, органы, стопы, процедуры, тесты и обратную связь. ИИ выполняет не роль, а загруженную карту допустимых переходов.

Если сказать проще: обычный промпт просит ИИ "будь полезным". Сократ-программа объясняет ИИ, какие состояния различать, какие переходы разрешены, что считать доказательством, где остановиться, кто владеет решением и как проверить результат.

От инструкции к физиологии



Дальше статья показывает полный маршрут: BOIS задает грамматику, SIMA восстанавливает действующие машины, BORIS собирает доменную физиологию, а SOCRATES/COKPAT превращает ее в исполнимый пакет для ИИ.

2. Мини-словарь без академического тумана

Эти слова нужны не ради терминологии, а чтобы не смешивать разные операции.

Термин	Простой смысл	Зачем нужен в продукте ИИ
BOIS	Грамматика организованной деятельности с последствиями при неполном знании.	Не дает ИИ выдавать догадку за факт; требует D/V/S, границ и проверки.
SIMA [субстрато-независимый машинный анализатор]	Аналитик: смотрит на уже существующее поведение и восстанавливает, какая машина там работает.	Помогает понять пользователей, процесс, риски, источники ошибок и реальные opers [оперы].
BORIS [рациональная операционно-интеллектуальная система бизнеса]	Внедритель: превращает BOIS в узкую рабочую физиологию домена.	Создает правила продукта: роли, стопы, состояние, процедуры, тесты, поверхность ответа.
SOCRATES/СОКПАТ	Инженерное правиловое программирование вычисляющей среды.	Загружает в ИИ не "личность", а карту расчета и исполнения.
Расчет	Воспроизводимый переход состояния через операцию, стоимость, память и обратную связь.	Ответ ИИ должен быть не потоком текста, а закрытием расчетного перехода.
UCE [универсальная вычисляющая среда]	Среда, способная исполнять разные архитектуры исследования и действия.	LLM, инструменты, файлы, память, люди и процедуры вместе.

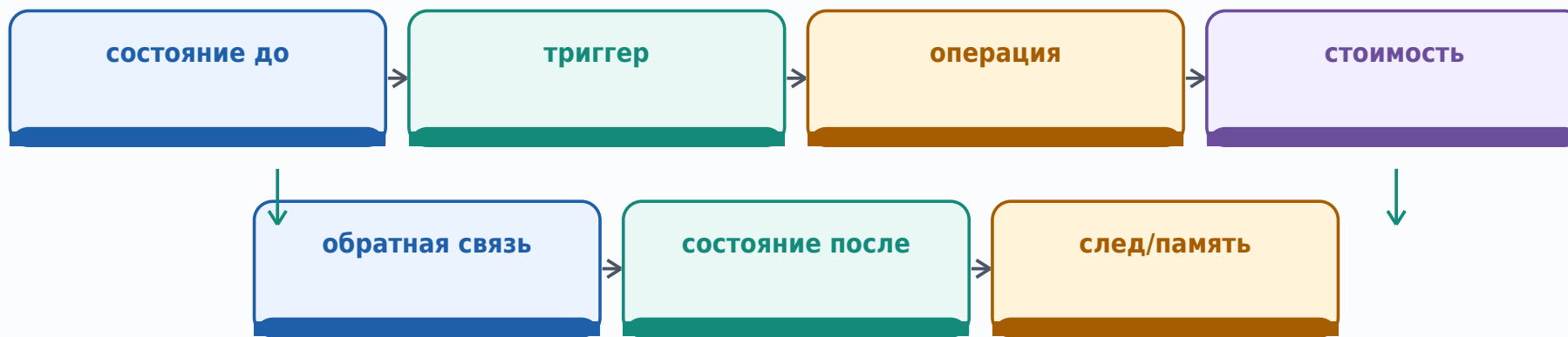
*i***Важно**

SIMA не ставит скрытый диагноз, BORIS не является вторым ядром BOIS, а SOCRATES/СОКПАТ не доказывает истину. Это инженерные роли внутри процесса.

3. Что значит "организовать расчет"

В бытовом языке расчет часто звучит как арифметика. В BOIS расчет шире: это воспроизводимое преобразование различных состояний. Было одно состояние, пришел триггер, выполнена операция, появилась цена, след, изменение и обратная связь.

расчет = переход



Для ИИ-продукта это означает: каждый ответ должен иметь место в переходе. ИИ не просто "говорит текст"; он различает исходное состояние, проверяет область, выбирает допустимый протокол, маркирует неизвестное, предлагает действие и показывает, что изменится после действия.

Обычный ответ vs расчетный ответ

Обычный ответ: "Я думаю, лучше сделать А".

Расчетный ответ: "В D/V/S сейчас неизвестно X; есть свидетельства Y; допустимый следующий шаг А является гипотезой изменения состояния; цена Z; стопы такие-то; если не сработает - fallback [резервный путь]".

4. Базовый цикл BOIS как цикл исследования

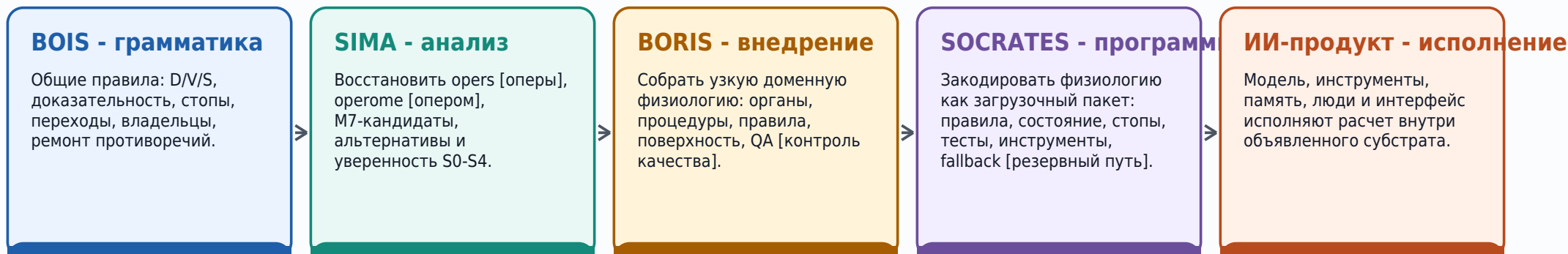
BOIS полезен там, где нельзя сразу дать чистую инструкцию: знаний мало, ошибка стоит дорого, субстрат сопротивляется, а действие имеет последствия. Тогда нужен не приказ "сделай правильно", а цикл исследования.



В хорошей ИИ-системе этот цикл становится невидимой дисциплиной продукта. Пользователь видит ясный ответ, но под ним работает расчет: что известно, что неизвестно, какая цена ошибки, кто владелец решения и где нужно остановиться.

5. SIMA&BORIS: две руки BOIS

SIMA&BORIS - это практический способ использовать BOIS. SIMA смотрит назад и вглубь: "какая машина уже действует?". BORIS смотрит вперед и в сборку: "какую физиологию нужно внедрить?". SOCRATES/СОКРАТ делает эту физиологию исполнимой для ИИ-среды.

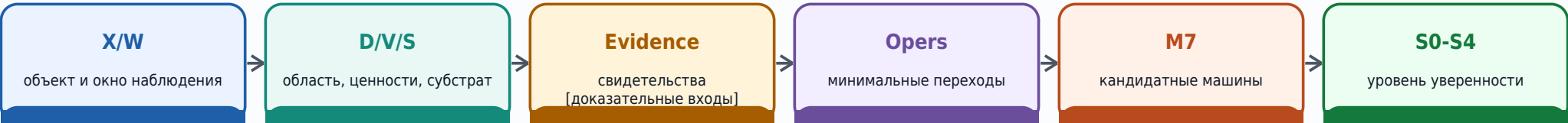


Если перепутать слои, продукт ломается. Нельзя использовать SIMA как оракул, BORIS как универсальную теорию, а SOCRATES как красивое название для промпта. У каждого слоя своя работа и своя граница.

6. SIMA: как понять машину до программирования

Перед тем как "учить ИИ", нужно понять, что именно он должен обслуживать. SIMA делает это без привязки к одному виду субстрата: процесс может быть цифровым, организационным, человеческим или гибридным.

Маршрут SIMA-реконструкции



Oper [опер] - атом практики

Oper выделяется только тогда, когда можно различить: состояние до, триггер, действие или расчет, агентность, свидетельство, изменение, память или остаток, стоимость и закрытие.

Элемент opers	Вопрос	Пример для ИИ-продукта
Состояние до	Что было различимо до действия?	Запрос пользователя, открытый кейс, уровень риска.
Триггер	Почему переход начался?	Пользователь просит решение, пришел документ, изменилась политика.
Операция	Что сделано?	Классификация, поиск источников, расчет риска, ответ.
Агентность	Кто действует и кто отвечает?	ИИ предлагает, человек утверждает, система логирует.
Свидетельство	На чем основан вывод?	Файл, политика, лог, дата, цитата, инструмент.
Стоимость	Что потрачено или поставлено под риск?	Время, деньги, доверие, приватность, ошибка.
Закрытие	Как понять, что переход завершен?	Ответ принят, эскалация создана, тест пройден.

7. BORIS: как превратить анализ в рабочую физиологию

BORIS берет результаты SIMA и собирает внедряемый контур. Слово business здесь понимается широко: организованное дело с целями, ролями, памятью, стоимостью, обратной связью и ответственностью.

Органы

кто за что отвечает: доказательства, риск, стиль, инструменты, эскалация.

Правила

какие переходы допустимы, что запрещено, что требует ремонта.

Состояние

что продукт помнит: кейс, источники, риск, владелец, статус.

Процедуры

как делать: загрузка, анализ, ответ, эскалация, сохранение.

Стопы

когда остановиться: нет полномочий, нет доказательств, высокий риск.

QA [контроль]

сценарии, регрессии, dry-runs, field checks.

?

BORIS-вопрос

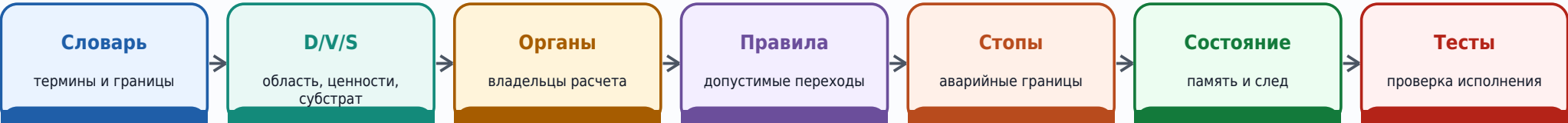
Не "как заставить модель отвечать красиво?", а "какая физиология должна удерживать продукт от ложного закрытия, скрытой власти, подмены источников и аварийных последствий?"

BORIS особенно важен для ИИ-продуктов, потому что языковая модель легко создает правдоподобную поверхность. Физиология нужна, чтобы под поверхностью были владельцы, состояние, доказательства, стопы и тесты.

8. Что значит программировать на SOCRATES/СОКРАТ для ИИ

Программировать на SOCRATES/СОКРАТ - значит задавать не строки кода в обычном смысле, а исполнимую физиологию для универсальной вычисляющей среды UCE. Для ИИ это особенно естественно: модель понимает текстовые правила, но ей нужно дать не только правила поведения, а карту расчета.

Минимальный стек Сократ-программы



В обычном программировании мы часто пишем функции. В Сократ-программировании для ИИ мы пишем "физиологию функций": кто имеет право запускать переход, какие данные нужны, что считается доказательством, какая цена, какие стопы, что записать в состояние и как проверить результат.

Обычный подход	Сократ-подход
"Ты эксперт, отвечай подробно".	"Ты выполняешь такую-то физиологию в D/V/S; факты, выводы и гипотезы разделены; стопы обязательны".
Тон и роль важнее состояния.	Состояние, владельцы и последствия важнее образа роли.
Тестируется красивость ответа.	Тестируется корректность перехода, stop behavior, доказательность и fallback.
Ошибки чинятся новым промптом.	Ошибки превращаются в новые правила, стопы, процедуры и тесты.

9. Полный маршрут: от идеи до ИИ-продукта

Ниже - практическая последовательность. Ее можно использовать для создания консультанта, аналитика документов, игрового мастера, ассистента поддержки, внутреннего аудитора, проектного помощника или другого ИИ-продукта.



Пошагово

- Объяви продуктовую область D: например, "анализ договоров поставки для внутреннего юриста", а не "юридический ИИ вообще".
- Задай ценности V: точность, безопасность, скорость, приватность, стоимость ошибки, горизонт ответственности.
- Опиши субстрат S: модель, файлы, инструменты, база знаний, память, люди, интерфейс, ограничения.
- Проведи SIMA: собери реальные кейсы, выдели ops, альтернативные машины и уровни уверенности S0-S4.
- Собери BORIS-физиологию: органы, правила, стопы, процедуры, state schema, tool policy.
- Скомпилируй SOCRATES-пакет: PDF-канон, JSON/CSV-представления, тесты, манифест, хэши, порядок загрузки.
- Запусти и проверь: сценарии, атаки, регрессии, dry-runs, затем ограниченный field pilot.

10. Как запрос пользователя проходит через Сократ-продукт

Пользователь видит простой интерфейс: написал запрос - получил ответ. Внутри хорошего Сократ-продукта проходит цепочка переходов.



Такой runtime не делает ответ тяжелым. Большая часть расчета может быть скрытой. Наружу выводятся только важные границы: что известно, что неизвестно, какие риски, что делать дальше и где человек должен принять решение.

R

Ключевое правило

Чем выше риск и стоимость ошибки, тем больше внутренний расчет должен выходить на поверхность: источники, допущения, STOP-сигналы, владельцы и fallback. В низком риске поверхность может быть компактной.

11. Анатомия Сократ-пакета для ИИ

Для серьезного ИИ-продукта одного системного сообщения мало. Нужен пакет, который можно загрузить, проверить, восстановить и отремонтировать. Ниже - минимальная анатомия.

Папка / слой	Что хранит	Зачем нужно
core/	PDF-канон, машинная карта, source hierarchy.	Человек и машина понимают одну норму.
tables/	Правила, термины, органы, стопы, процедуры, тесты, критерии.	Физиология становится проверяемой, а не устной.
runtime/	Движок хода, tool policy, surface policy.	ИИ знает, как исполнять запрос в конкретной среде.
state/	State schema, память, события, журнал дельт, save template.	Продукт не теряет контекст и не смешивает уровни.
qa/	Сценарии, регрессии, adversarial tests, dry-runs.	Ошибки ловятся до пользователя.
integrity/	Manifest, hashes, validation report.	Пакет можно проверить и не подменить незаметно.

!

PDF и JSON не конкурируют

PDF остается человекочитаемой канонической поверхностью; JSON/CSV помогают машине исполнять и проверять. Если они расходятся, это не "вариант нормы", а дефект пакета.

12. M7: как описывать философскую машину продукта

Когда SIMA восстанавливает машину, а BORIS собирает физиологию, удобно держать M7-карту. Она не превращает продукт в философский трактат; она помогает не забыть семь важных сторон.

O - Ontology [онтология]

что существует: пользователи, документы, кейсы, роли, состояния.

E - Epistemology [эпистемология]

что считается знанием, свидетельством, гипотезой, слухом.

A - Agency [агентность]

кто действует, кто утверждает, кто несет ответственность.

V - Values [ценности]

какая цена недопустима, что важнее при конфликте.

R - Risk [риск]

что может сломаться: юридически, финансово, технически, этически.

C - Closure [закрытие]

как понять, что переход завершен и что осталось долгом.

T - Transition [переход]

какие изменения состояния допустимы и проверяемы.

Для ИИ-продукта M7 особенно полезна против "магической роли". Модель не должна просто играть эксперта; она должна исполнять переходы внутри онтологии, эпистемологии, агентности, ценностей, риска, закрытия и переходной грамматики.

13. Как "кодить" на Сократе: минимальный модуль

Ниже - не синтаксис конкретного языка, а практический шаблон. Его можно хранить в YAML/JSON, таблице или PDF-каноне. Главное - чтобы ИИ мог загрузить эту физиологию и чтобы человек мог ее проверить.

```
module: ai_product.case_triage
ru_name: "Разбор входящего кейса"
D: "Внутренний анализ обращений клиентов по действующей политике"
V:
  - "Не обещать то, на что нет полномочий"
  - "Разделять факт, вывод, гипотезу и совет"
S:
  model: "LLM + поиск по базе + журнал кейса + человек-утверждающий"
  limits: ["политики могут устареть", "не все документы доступны"]
  state:
    fields: [case_id, user_request, sources, risk_level, owner, status]
  organs:
    ORG-EVIDENCE: "проверяет источники"
    ORG-RISK: "классифицирует риск"
    ORG-AUTHORITY: "проверяет полномочия"
  opers:
    - id: OPER-CLASSIFY
      trigger: "новый запрос"
      action: "классифицировать тему, риск и недостающие данные"
  stops: [STOP-NO-AUTHORITY, STOP-EVIDENCE-GAP]
```

C

Что здесь программируется?

Не "личность ассистента", а переход: какие данные принять, как их проверить, кто владелец, что записать, где остановиться и как закрыть кейс.

14. Загрузочная инструкция для ИИ: не промпт, а контракт

Когда Сократ-пакет попадает в ИИ-среду, стартовое сообщение должно не вдохновлять модель, а фиксировать контракт исполнения.

```
Ты выполняешь загруженную физиологию продукта <NAME>.  
Источник канона: core/<PRODUCT>.pdf.  
Машинные поверхности: tables/*.csv, runtime/*.json, state/*.json.  
Перед ответом:  
1. Определи D/V/S запроса.  
2. Проверь, есть ли активные STOP-сигналы.  
3. Раздели факт, вывод, гипотезу, оценку, риск и неизвестное.  
4. Используй инструменты только по tool policy.  
5. Не финализируй решение, если нет полномочий владельца.  
6. Закрой ответ: next_step, state_delta, open_debts, fallback.  
Если правила конфликтуют:  
- не выбирай удобную сторону;  
- включи STOP-RULE-CONFLICT;  
- предложи ремонт физиологии.
```

Это все еще может быть загружено как текст. Но смысл не в тексте как таковом, а в том, что текст ссылается на проверяемую физиологию: таблицы, схемы, тесты, стопы, владельцев и состояние.

15. Пример: ИИ-аналитик документов

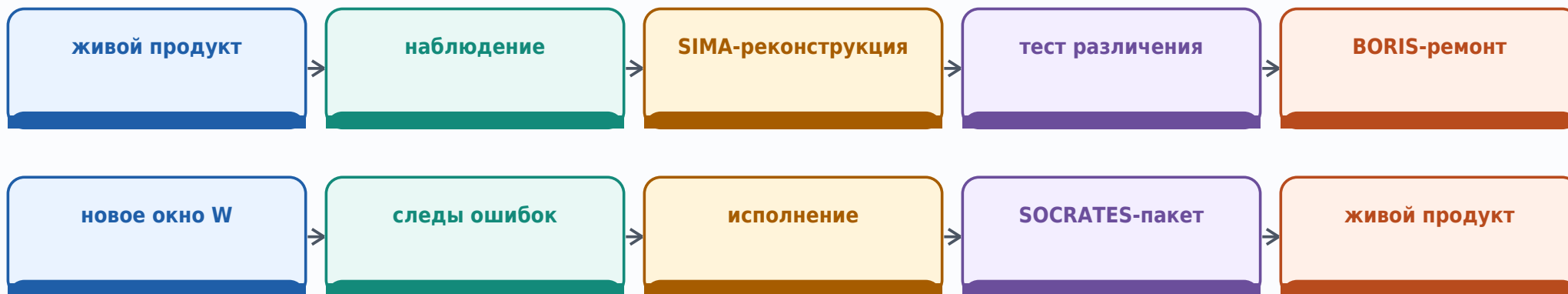
Представим продукт: ИИ помогает команде быстро разбирать договоры поставки. Без SOCRATES/СОКРАТ это легко превращается в "прочитай договор и скажи риски". С SOCRATES/СОКРАТ мы строим физиологию.



Слой	Решение для продукта
D	Только внутренний предварительный анализ договоров поставки; не юридическое заключение.
V	Не придумывать норму; цитировать источник; эскалировать высокий риск; не обещать исход переговоров.
S	LLM, PDF-договоры, поиск по политике, журнал кейса, человек-юрист.
Органы	ORG-SOURCE, ORG-RISK, ORG-AUTHORITY, ORG-REDACTION, ORG-STYLE.
Стопы	Нет текста договора; источник конфликтует; высокий юридический риск; пользователь просит финальное заключение.
Выход	Карта рисков: цитата, риск, уровень уверенности, владелец, предлагаемое следующее действие.

16. SIMA&BORIS как цикл ремонта продукта

ИИ-продукт нельзя "один раз написать" и забыть. Пользователи, документы, регуляторика, инструменты и ожидания меняются. Поэтому SIMA&BORIS работают циклом.



нижний ряд показывает обратный путь: новое окно W -> следы ошибок -> исполнение -> SOCRATES-пакет -> живой продукт

После каждого сбоя не нужно просто добавлять длинную фразу в промпт. Нужно спросить: какая часть физиологии ошиблась? Словарь? Орган? Стоп? Состояние? Политика инструмента? Тест? Поверхность ответа?

R

Правило ремонта

Сбой пользователя превращается не в "модель была плохая", а в один из ремонтируемых объектов: термин, правило, стоп, процедура, test case, state field, owner или fallback.

17. QA [контроль качества] для Сократ-продукта

Контроль качества должен проверять не только правильные ответы, но и правильные остановки, границы полномочий, устойчивость к обману, восстановление состояния и отсутствие скрытых ярлыков.

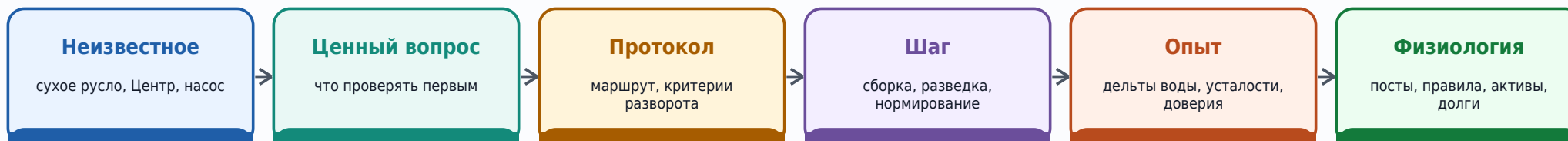
Тип проверки	Что ловит	Мини-пример
Schema test	Сломанные поля состояния, несовместимые версии.	state.owner обязателен, risk_level только из списка.
Cross-reference	Правило ссылается на несуществующий стоп или орган.	RULE-17 -> STOP-X, которого нет.
Adversarial	Пользователь просит обойти полномочия или скрыть риск.	"Скажи клиенту, что возврат одобрен".
Metamorphic	Ответ не должен меняться от несущественной переформулировки.	Один и тот же риск в двух стилях запроса.
Dry-run	Динамика на нескольких шагах без реальных пользователей.	Кейс проходит 5 переходов, проверяются долги.
Field pilot	Реальная применимость и человеческая оценка.	Ограниченный запуск с логами и независимым ревью.

**Не завышать QA**

Если были только структурные и сценарные проверки, нельзя писать "доказана полевая эффективность". QA-E требует реальных пользователей, задач, логов и независимой оценки.

18. Почему игра MaOS важна как педагогика BOIS

Игровая физиология MaOS показывает, как обучать BOIS ненавязчиво. Игроку не читают лекцию про D/V/S, SIMA или протокол. Он сталкивается с водой, куполом, усталостью, Центром, советником, долгами и отскоками. Так BOIS становится пережитой практикой.



Эта же логика нужна ИИ-продуктам. Пользователь не обязан знать BOIS-словарь; продукт должен вести его через ясные переходы, показывать последствия, не подменять факт надеждой и превращать опыт в устойчивую практику.

Р

Педагогическое правило

Хороший SOCRATES-продукт обучает не лозунгами, а структурой взаимодействия: что уточняется, что доказывается, что нельзя обещать, где нужен владелец, что стало активом и какой долг остался.

19. Типовые провалы при программировании ИИ на SOCRATES/SOKPAT

Мега-промпт

много текста, но нет состояния, тестов, владельцев и стопов.

Оракул

ИИ выбирает за владельца и скрывает неопределенность.

Красивый ответ

тон заменяет доказательства и расчет.

Скрытый ярлык

шаблон, score или статус подменяет смысловой расчет.

Tool chaos

инструменты доступны без политики, журналов и fallback.

QA только happy path

проверяются только хорошие запросы, стопы не тестируются.

Ложная универсальность

частный продукт объявляет себя решением для всего.

Нет ремонта

ошибки накапливаются как исключения, а не превращаются в физиологию.

Почти все эти провалы выглядят как "модель ошиблась". Но с точки зрения BOIS часто ошиблась не модель, а физиология: ей не дали границу, состояние, стоп, владельца или тест.

20. Как внедрять в команде

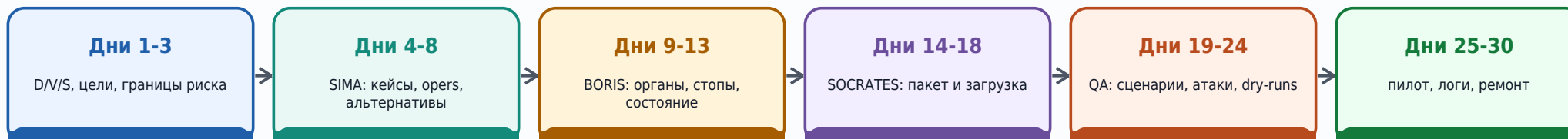
SOCRATES/COKPAT не требует, чтобы вся команда стала философами. Он требует, чтобы продуктовые решения стали видимыми и проверяемыми.

Роль команды	Что делает в SIMA&BORIS	Артефакт
Product owner [владелец продукта]	Фиксирует D/V/S, ценности, границы и критерии пользы.	Product D/V/S card.
Domain expert [эксперт домена]	Помогает выделять opers, исключения, источники и недопустимые ошибки.	Operome, risk map.
AI engineer [ИИ-инженер]	Собирает runtime, инструменты, схемы состояния и загрузку.	SOCRATES package.
QA / auditor [аудитор]	Проверяет стопы, противоречия, регрессии и полевую границу.	Validation report.
Human approver [человек-утверждающий]	Принимает решения там, где у ИИ нет полномочий.	Authority log.

Минимальный ритм работы: один раз собрать исходную физиологию, затем после каждого набора тестов или полевых сбоев проводить короткий SIMA-разбор и BORIS-ремонт.

21. Практический план на 30 дней

Ниже - реалистичный план для команды, которая хочет построить первый ИИ-продукт в стиле SOCRATES/СОКРАТ.



Готовность к пилоту

- Есть канон: что продукт делает и чего не делает.
- Есть state schema и журнал переходов.
- Есть стопы для высокого риска, неизвестного, отсутствия полномочий и конфликта источников.
- Есть минимум 20 тестовых сценариев, включая атаки и неоднозначные запросы.
- Есть человек-владелец для решений, которые ИИ не должен финализировать.
- Есть процедура ремонта: как ошибка превращается в правило, тест или изменение состояния.

22. Одностраничная рабочая карта

Эту карту можно использовать перед созданием любого ИИ-продукта.

Вопрос	Ответ должен быть записан
1. Что такое D?	Точная область продукта, не "все для всех".
2. Что такое V?	Цели, запретные цены, риск, горизонт ответственности.
3. Что такое S?	Модель, инструменты, данные, люди, память, ограничения.
4. Что уже работает?	SIMA: opers, operome, M7, альтернативы, S0-S4.
5. Что внедряем?	BORIS: органы, правила, стопы, процедуры, состояние, поверхность.
6. Как кодируем?	SOCRATES: PDF-канон, JSON/CSV, манифест, хэши, порядок загрузки.
7. Как исполняем?	Runtime: запрос -> проверка -> расчет -> ответ -> дельта состояния.
8. Как проверяем?	QA: схемы, ссылки, стопы, регрессии, сухие прогоны, пилот.
9. Как ремонтируем?	Ошибка -> термин/правило/стоп/процедура/тест/state field.

Σ

Формула

SIMA видит машину. BORIS собирает физиологию. SOCRATES загружает физиологию в ИИ. BOIS удерживает весь цикл от ложного закрытия, смешения слоев и непроверенного переноса.

23. Финальная формула

Программировать на SOCRATES/COKPAT для ИИ через BOIS: SIMA&BORIS - значит строить не "умного болтуна", а вычисляющую физиологию действия при неполном знании.

1

В одной фразе

ИИ-продукт становится зрелым не тогда, когда он красиво отвечает, а когда он умеет различать состояние, доказательство, риск, полномочие, цену, стоп и следующий проверяемый переход.

Что должен унести читатель

- BOIS задает грамматику: как действовать с последствиями при неполном знании.
- SIMA помогает понять уже действующую машину и не выдумать продукт из головы.
- BORIS превращает понимание в рабочую доменную физиологию.
- SOCRATES/COKPAT программирует ИИ через правила, состояние, органы, стопы, процедуры, тесты и обратную связь.
- Промпт может быть частью загрузки, но не заменяет физиологию.
- Каждый сбой продукта должен возвращаться в SIMA&BORIS-ремонт, а не просто замазываться новой фразой.

Эта статья является учебным объяснением подхода. Она не заявляет полевую эффективность конкретного продукта без отдельного QA-E [полевой проверки].